

Name: Bùi Quang Minh  
Class: DS66B

## Lab 1: Experiencing Blocking (Row-Level Lock)

The image displays two screenshots of the pgAdmin 4 interface, illustrating a blocking scenario in PostgreSQL.

**Left Screenshot:** The interface shows a query window with the following SQL code:

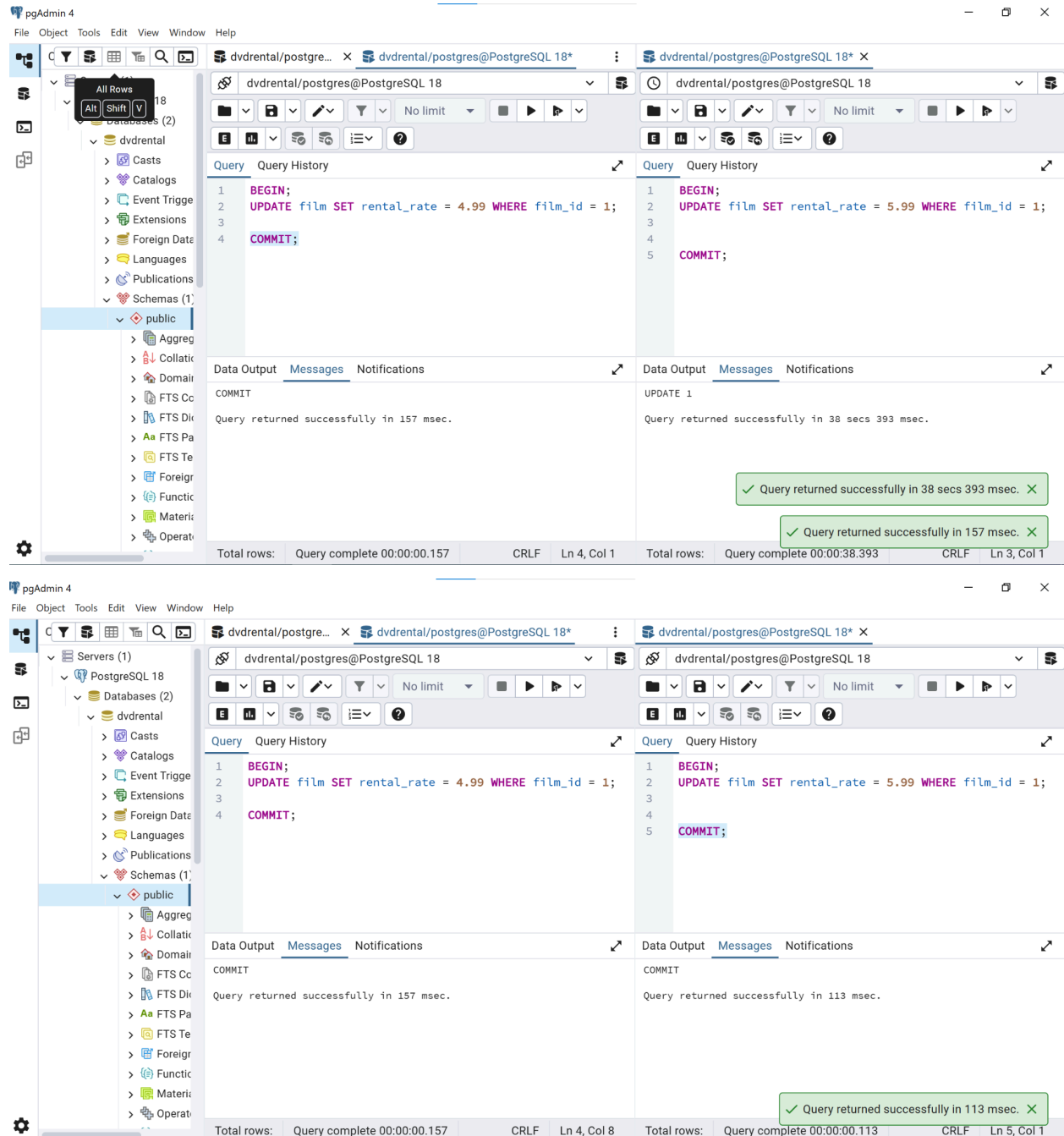
```
1 BEGIN;  
2 UPDATE film SET rental_rate = 4.99 WHERE film_id = 1;  
3  
4 COMMIT;
```

The query is executed successfully, as indicated by the message: "Query returned successfully in 206 msec."

**Right Screenshot:** The interface shows a second query window with the following SQL code:

```
1 BEGIN;  
2 UPDATE film SET rental_rate = 5.99 WHERE film_id = 1;  
3  
4  
5 COMMIT;
```

The query is blocked, as indicated by the message: "Query returned successfully in 229 msec." The status bar at the bottom of the window shows: "Total rows: Waiting for the query to complete... 00:00:09.680".



In this lab, we observe how Row-level Exclusive Locks (X-Locks) block other writers but not readers. When Transaction A updates a row, it acquires an Exclusive Lock (X-Lock). If Transaction B attempts to update the exact same row, it will be blocked immediately and must wait until Transaction A commits or rollbacks to release the lock.

## Lab 2: Replicating Non-repeatable Read

pgAdmin 4

File Object Tools Edit View Window Help

Servers (1)  
PostgreSQL 18  
Databases (2)  
dvdrental  
public

Query Query History

```
1 BEGIN;  
2 SELECT rental_rate FROM film  
3 WHERE film_id = 10;  
4  
5  
6 select rental_rate FROM film  
7 WHERE film_id = 10;  
8 COMMIT;
```

Data Output Messages Notifications

| rental_rate |
|-------------|
| 4.99        |

Showing rows: 1 to 1 Page No: 1 of 1

Total rows: 1 Query complete 00:00:00.856 CRLF Ln 1, Col 1

Query returned successfully in 113 msec.

Successfully run. Total query runtime: 856 msec. 1 rows affected.

pgAdmin 4

File Object Tools Edit View Window Help

Servers (1)  
PostgreSQL 18  
Databases (2)  
dvdrental  
public

Query Query History

```
1 BEGIN;  
2 SELECT rental_rate FROM film  
3 WHERE film_id = 10;  
4  
5  
6 select rental_rate FROM film  
7 WHERE film_id = 10;  
8 COMMIT;
```

Data Output Messages Notifications

| rental_rate |
|-------------|
| 4.99        |

Showing rows: 1 to 1 Page No: 1 of 1

Total rows: 1 Query complete 00:00:00.856 CRLF Ln 1, Col 1

Query returned successfully in 103 msec.

Query returned successfully in 103 msec.

The image displays two screenshots of the pgAdmin 4 interface, illustrating a database update and a subsequent read query.

**Top Screenshot:** The left pane shows the database structure with 'dvdrental' selected under 'PostgreSQL 18'. The 'public' schema is expanded, showing tables like 'film'. The main pane shows a SQL query window with the following query:

```
BEGIN;  
SELECT rental_rate FROM film  
WHERE film_id = 10;  
  
select rental_rate FROM film  
WHERE film_id = 10;  
COMMIT;
```

The 'Data Output' tab shows the result of the query:

| rental_rate |
|-------------|
| 4.99        |

The status bar indicates 'Total rows: 1' and 'Query complete 00:00:00.856'.

**Bottom Screenshot:** The same pgAdmin 4 interface is shown, but the SQL query window now contains a different query:

```
BEGIN;  
UPDATE film SET rental_rate = 0.99 WHERE film_id = 10;  
  
COMMIT;
```

The 'Data Output' tab shows the result of the query:

| rental_rate |
|-------------|
| 0.99        |

The status bar indicates 'Total rows: 1' and 'Query complete 00:00:00.137'.

A green notification box at the bottom right of the bottom screenshot states: 'Successfully run. Total query runtime: 137 msec. 1 rows affected.'

A Non-repeatable Read occurs when a transaction (Session A) reads the exact same row twice, but gets two different results because another transaction (Session B) updated that row and committed in between. The error here is the inconsistency from Session A's perspective during its lifespan.

### Lab 3: Creating a Deadlock

pgAdmin 4

File Object Tools Edit View Window Help

Servers (1)  
PostgreSQL 18  
Databases (2)  
dvdrental  
Catalogs  
Event Triggers  
Extensions  
Foreign Data  
Languages  
Publications  
Schemas (1)  
public  
Aggregates  
Collations  
Domains  
FTS Configuration  
FTS Templates  
Foreign Data Wrappers  
Functions  
Materialized Views  
Operators

Query Query History

```
1 BEGIN;  
2 UPDATE film SET rental_rate = 2.99 WHERE film_id = 100;  
3  
4 UPDATE film SET rental_rate = 3.99 WHERE film_id = 200;
```

Data Output Messages Notifications

UPDATE 1  
Query returned successfully in 127 msec.

Total rows: Query complete 00:00:00.127 CRLF Ln 2, Col 56

Query Query History

```
1 BEGIN;  
2 UPDATE film SET rental_rate = 2.99 WHERE film_id = 200;  
3  
4 UPDATE film SET rental_rate = 3.99 WHERE film_id = 100;
```

Data Output Messages Notifications

COMMIT  
Query returned successfully in 161 msec.

Total rows: Query complete 00:00:00.161 CRLF Ln 4, Col 55

✓ Query returned successfully in 127 msec. ✕

pgAdmin 4

File Object Tools Edit View Window Help

Servers (1)  
PostgreSQL 18  
Databases (2)  
dvdrental  
Catalogs  
Event Triggers  
Extensions  
Foreign Data  
Languages  
Publications  
Schemas (1)  
public  
Aggregates  
Collations  
Domains  
FTS Configuration  
FTS Templates  
Foreign Data Wrappers  
Functions  
Materialized Views  
Operators

Query Query History

```
1 BEGIN;  
2 UPDATE film SET rental_rate = 2.99 WHERE film_id = 100;  
3  
4 UPDATE film SET rental_rate = 3.99 WHERE film_id = 200;
```

Data Output Messages Notifications

UPDATE 1  
Query returned successfully in 127 msec.

Total rows: Query complete 00:00:00.127 CRLF Ln 2, Col 56

Query Query History

```
1 BEGIN;  
2 UPDATE film SET rental_rate = 2.99 WHERE film_id = 200;  
3  
4 UPDATE film SET rental_rate = 3.99 WHERE film_id = 100;
```

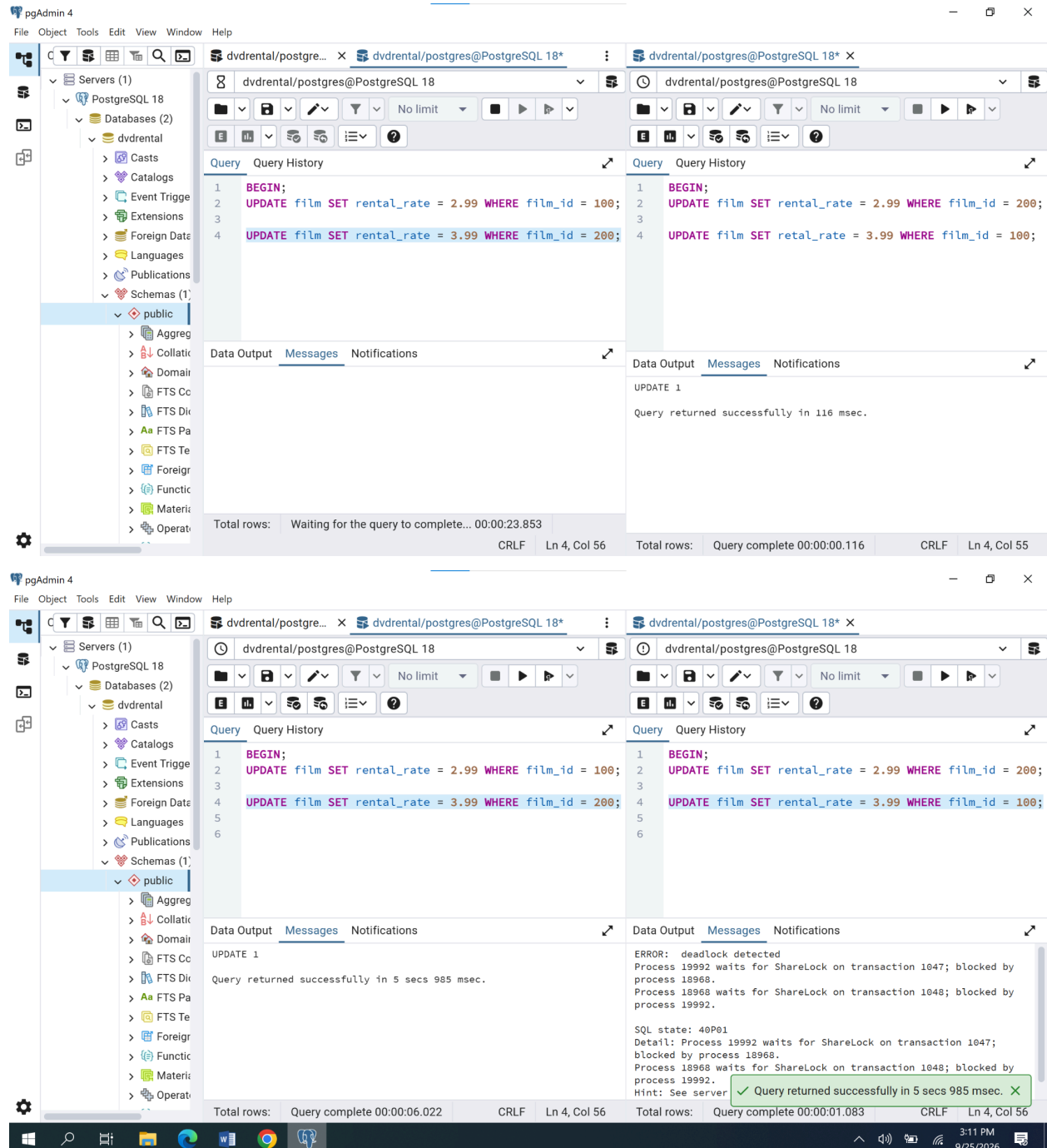
Data Output Messages Notifications

UPDATE 1  
Query returned successfully in 116 msec.

Total rows: Query complete 00:00:00.116 CRLF Ln 2, Col 56

✓ Query returned successfully in 116 msec. ✕

3:03 PM  
9/25/2026



A Deadlock occurs when two or more transactions are waiting for each other to release locks, resulting in an infinite wait state. PostgreSQL uses Deadlock Detection instead of prevention. When a circular wait is detected (Session A waits for B, and B waits for A), PostgreSQL will choose a victim and abort it so the other transaction can proceed.

## Lab 4: Preventing Race Conditions (FOR UPDATE)

pgAdmin 4

File Object Tools Edit View Window Help

dvdrental/postgres@PostgreSQL 18\*

dvdrental/postgres@PostgreSQL 18\*

Query Query History

```
1 BEGIN;
2 SELECT * FROM inventory
3 WHERE inventory_id = 10 FOR UPDATE;
4
5 INSERT INTO rental (rental_date, inventory_id, customer_id)
6 VALUES (NOW(), 10, 1, 1);
7 COMMIT;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

| inventory_id [PK] integer | film_id smallint | store_id smallint | last_update timestamp without time zone |
|---------------------------|------------------|-------------------|-----------------------------------------|
| 1                         | 10               | 2                 | 2006-02-15 10:09:17                     |

pgAdmin 4

File Object Tools Edit View Window Help

dvdrental/postgres@PostgreSQL 18\*

dvdrental/postgres@PostgreSQL 18\*

Query Query History

```
1 BEGIN;
2 SELECT * FROM inventory
3 WHERE inventory_id = 10 FOR UPDATE;
4
5 INSERT INTO rental (rental_date, inventory_id, customer_id)
6 VALUES (NOW(), 10, 1, 1);
7 COMMIT;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

| inventory_id [PK] integer | film_id smallint | store_id smallint | last_update timestamp without time zone |
|---------------------------|------------------|-------------------|-----------------------------------------|
| 1                         | 10               | 2                 | 2006-02-15 10:09:17                     |

Total rows: Waiting for the query to complete... 00:00:05.780

The image displays two screenshots of the pgAdmin 4 interface, showing SQL queries and their results in a PostgreSQL database.

**Top Screenshot:**

- Left Panel:** Shows the database structure. The 'public' schema is selected, and the 'rental' table is highlighted.
- Query Editor:** Contains the following SQL query:

```
1 BEGIN;  
2 SELECT * FROM inventory  
3 WHERE inventory_id = 10 FOR UPDATE;  
4  
5 INSERT INTO rental (rental_date, inventory_id, customer_id,  
6 VALUES (NOW(), 10, 1, 1);  
7 COMMIT;  
8  
9
```
- Messages Panel:** Shows the query execution results:

```
COMMIT  
Query returned successfully in 214 msec.
```
- Table View:** Displays the 'rental' table with the following data:

| inventory_id [PK] integer | film_id smallint | store_id smallint | last_update timestamp without time zone |
|---------------------------|------------------|-------------------|-----------------------------------------|
| 10                        | 2                | 2                 | 2006-02-15 10:09:17                     |
- Status Bar:** Shows 'Successfully run. Total query runtime: 20 secs 995 msec. 1 rows affected.'

**Bottom Screenshot:**

- Left Panel:** Same as the top screenshot, showing the 'public' schema and 'rental' table.
- Query Editor:** Contains the following SQL query:

```
1 BEGIN;  
2 SELECT * FROM inventory  
3 WHERE inventory_id = 10 FOR UPDATE;  
4  
5 INSERT INTO rental (rental_date, inventory_id, customer_id,  
6 VALUES (NOW(), 10, 1, 1);  
7 COMMIT;  
8  
9
```
- Messages Panel:** Shows the query execution results:

```
ROLLBACK  
COMMIT  
Query returned successfully in 136 msec.
```
- Status Bar:** Shows 'Query returned successfully in 136 msec.'

To prevent Race Conditions in real-world scenarios, such as two customers trying to rent the exact same DVD copy simultaneously, we use Pessimistic Locking. By explicitly using the `SELECT ... FOR UPDATE` statement, developers can manually acquire an X-Lock before updating, ensuring that no other transactions can modify the data while it is being processed.